

TEST REPORT

FirmTrack — TimeTrack & AccountTrack

Verification of TimeTrack enhancements, AccountTrack Phase 1 & Phase 2 (Chart of Accounts, General Ledger, Registers, Budgeting), and the automated regression suite

Prepared for: FirmTrack (Compnet Final)

Prepared by: Claude Code (development & QA session)

Report date: 11 July 2026

Scope: TimeTrack matter-summary/billable/calendar features; AccountTrack Phase 1 (invoices, disbursements, trust ledger); AccountTrack Phase 2 Stage 1–3 (Registers, double-entry General Ledger, period closing, budgeting); automated Vitest regression suite.

1. Executive Summary

All features covered in this report were verified end-to-end using real browser automation (Playwright) against the running application and a real Supabase/Postgres database — not typechecking or code review alone. Testing was iterative: every defect found was fixed and re-verified before being marked passed. A permanent automated regression suite (Vitest) was subsequently built to protect the General Ledger logic — the one part of the app that moves real money — against future regressions.

6 FEATURE PHASES VERIFIED	3 REAL BUGS FOUND & FIXED	18 / 18 AUTOMATED TESTS PASSING	0 KNOWN OPEN DEFECTS
------------------------------	------------------------------	------------------------------------	-------------------------

Bottom line: every phase tested ended in a fully passing state with zero console errors and zero failed network requests in the final verification pass. Three genuine bugs were caught during manual testing (two schema/infrastructure issues, one business-logic deadlock in period reopening) — all three were root-caused, fixed, and re-verified. The AccountTrack General Ledger now has 18 automated regression tests covering balance/atomicity guarantees, all four auto-posting trigger points, period closing/reopening (including real closing-entry computation), statement reconciliation, and aggregation correctness for the Matter Register and Lawyer Overview.

2. Test Coverage Overview

Phase	Feature area	Method	Result
TimeTrack enhancements	Billable/non-billable flag, calendar-event linking, matter summary card, print view	Playwright, real login	FIXED & PASSED
AccountTrack Phase 1	Invoices, disbursements, trust/retainer ledger, accounts role, matter budget bare UI	Playwright, real login	FIXED & PASSED
AccountTrack Phase 2 — Stage 1	Time/Trust/Expense/Invoice/Matter Registers	Playwright, real login	PASSED
AccountTrack Phase 2 — Stage 2	Chart of Accounts, double-entry General Ledger, auto-posting, Statements, period closing	Playwright, real login	FIXED & PASSED
AccountTrack Phase 2 — Stage 3	Lawyer Overview, budgeting (lawyer + matter)	Playwright, real login	PASSED
Automated regression suite	GL posting logic, period locking, statements, register aggregation	Vitest integration tests	18 / 18 PASSED

3. Defects Found & Resolved

#	Defect	Root cause	Resolution
1	TimeTrack save / Activity Log "Convert to entry" failing against production	The <code>time_entries</code> table pre-dated the migrations folder; <code>create table if not exists</code> silently no-op'd against the pre-existing legacy table, so the intended columns (<code>lawyer_id</code> , <code>matter_id</code> , <code>rate_usd</code> , <code>status</code> , etc.) were never created. A second, deeper issue then surfaced: the legacy table also had <code>NOT NULL</code> constraints on <code>user_id</code> / <code>matter</code> / <code>hours</code> that the current app never populates.	Two migrations: added the missing columns with best-effort backfill from legacy data, then a follow-up migration relaxed the leftover <code>NOT NULL</code> constraints.
2	Matter Summary print view rendered a blank page	Stale Turbopack dev-server cache was serving an earlier draft of the print CSS (a <code>:has()</code> / <code>:not()</code> selector approach) instead of the corrected <code>visibility</code> -based version already saved to disk.	Cleared the <code>.next</code> build cache and restarted the dev server; confirmed the correct CSS was then served and the print preview showed only the intended card.
3	Reopening a closed accounting period deadlocked	The reversal journal entry was dated inside the period being reopened, and <code>post_journal_entry</code> 's own period-closed check rejected it — a closed period couldn't be reversed <i>because</i> it was still closed.	Reordered the reopen route: flip the period's status to <code>open</code> first, then post the reversal entry, which is then accepted.

All three defects were caught during first-pass manual verification, not discovered later in production — the value of running each feature through a real browser/API flow before declaring it complete.

4. Detailed Results — TimeTrack Enhancements

4.1 Scope

- Billable / Non-Billable selector on TimeTrack and Quick Fee rows
- Calendar-event linking (CalenTrack event picker prefilling date/hours/matter)
- Matter Summary Card (Billable/Non-Billable/Total Hours/Amount, Matter/Client/Name/Re Line, Unbilled Disbursements/Hours/Fees, A/R/Retainer/Trust)
- Print view of the Matter Summary Card

4.2 Method

Logged into the live app with a real test account via Playwright; created a client, matter, and a mix of billable/non-billable time entries through the actual UI and API; filtered Saved Entries by matter to trigger the summary card; emulated print media and screenshotted the result.

4.3 Result

PASS (after fixing Defects #1 and #2 above). Final verification: 2h billable + 1h non-billable entries produced a summary card showing Non-Billable 1.00h, Billable 2.00h, Total 3.00h, correct Unbilled Hours, and A/R/Trust reflecting subsequently-added disbursement and trust-ledger data. Print preview correctly isolated the card from the rest of the page. Zero console errors, zero failed network requests in the final pass.

5. Detailed Results — AccountTrack Phase 1

5.1 Scope

- `invoices`, `disbursements`, append-only `trust_ledger_entries` tables
- Per-matter data-entry page: add disbursement, add trust/retainer ledger entry, create invoice from unbilled items, record payment
- New `accounts` staff role with tiers, modeled on the existing lawyer/lawyer-category pattern

5.2 Method

End-to-end Playwright run: created a test client/matter, logged 2 time entries, added a \$150 disbursement and a \$1,000 trust deposit, created an invoice from the unbilled entry + disbursement, and confirmed the resulting numbers on both the AccountTrack page and the TimeTrack Matter Summary Card.

5.3 Result

PASS. Invoice `INV-2026-0001` correctly totalled \$150 (fees \$0 + disbursements \$150, since the time entry itself carried no rate). Unbilled Hours and Unbilled Disbursements correctly dropped to zero once billed; A/R correctly showed \$150 outstanding; Trust balance remained untouched at \$1,000. All arithmetic reconciled exactly across both surfaces that display it.

6. Detailed Results — AccountTrack Phase 2, Stage 1 (Registers)

6.1 Scope

Five tenant-wide, read-only register views: Time, Trust, Expense, Invoice, and Matter — the last backed by a new bulk-aggregation route designed to avoid N+1 queries as matter counts grow.

6.2 Method

Loaded each of the five register tabs in sequence via Playwright; exercised the Matter Register's search and status filters; cross-checked its aggregated figures against the already-verified per-matter Matter Summary Card for the same matter.

6.3 Result

PASS. All five tabs rendered without error. The Matter Register correctly listed every matter in the tenant — including a pre-existing real matter never touched by test scripts — confirming the view is genuinely tenant-wide rather than scoped to test data. Aggregated figures (4.00 billable hrs, 2.00 non-billable hrs, \$300 A/R, \$2,000 Trust) matched the per-matter summary exactly.

7. Detailed Results — AccountTrack Phase 2, Stage 2 (General Ledger)

7.1 Scope

- Chart of Accounts — 8 seeded default accounts (Operating Cash, Trust Bank, A/R, Costs Advanced to Clients, Trust Liability, Retainer Liability, Fees Earned, Retained Earnings) plus custom accounts
- Double-entry auto-posting on all four money-movement triggers: disbursements, trust deposits/ withdrawals, retainer deposits/withdrawals, and invoice creation/payment/void
- Income Statement and Balance Sheet, with Month/Year-to-Date and prior-period presets
- Accounting period closing (month = soft lock, year = real GAAP closing entry into Retained Earnings) and reopening (posts an exact reversal)
- A single Postgres function, `post_journal_entry`, guaranteeing every posting is atomic and balanced — the one deliberate exception to this codebase's otherwise transaction-free convention

7.2 Method

Two dedicated verification passes:

1. **Core posting & statements:** posted a disbursement and a trust deposit, confirmed both produced balanced journal entries (debit = credit in every case), confirmed the Balance Sheet's Assets = Liabilities + Equity self-check held, then closed the current month and confirmed a new posting into it was rejected with a 400 and the exact expected error, then reopened it and confirmed posting succeeded again. Also closed a historical year with no activity and confirmed no spurious closing entry was created.
2. **Real year-end closing entry:** posted a manual \$900 revenue entry dated in 2024, confirmed the Balance Sheet showed it as live "Current Year Earnings," closed 2024 as a year, confirmed a real closing entry was generated and Retained Earnings correctly absorbed the \$900 while Current Year Earnings dropped to \$0 (balance sheet still balanced), then reopened 2024 and confirmed the reversal restored the exact original split.

7.3 Result

FIXED & PASS. Defect #3 (the reopen deadlock) was found and fixed during this pass. After the fix, every assertion in both verification passes held exactly, including the GAAP-correct closing/reopening round-trip.

8. Detailed Results — AccountTrack Phase 2, Stage 3 (Budgeting & Lawyer Overview)

8.1 Scope

- Lawyer Overview: hours, revenue, WIP, and utilization per lawyer for a selected date range
- Lawyer budgets (target hours/billable hours/revenue) driving budget-relative utilization
- Matter budgets (cost ceiling) surfaced as a progress bar on the per-matter AccountTrack page

8.2 Method

Created a lawyer and logged 5 billable hours against a matter; verified utilization fell back to a logged-hours ratio with no budget set; set a 10-hour billable-hour budget target and confirmed utilization switched to a budget-relative calculation. Separately, set a \$400 budget ceiling on a matter already carrying \$500 of unbilled work via the real UI (not just the API) and confirmed the progress bar rendered correctly.

8.3 Result

PASS. Utilization computed exactly 50% (5 of 10 target billable hours) once a budget existed, correctly labeled as budget-relative rather than a logged-hours ratio. The matter budget bar correctly rendered in red with an "Over budget" label once \$500 of actual spend exceeded the \$400 ceiling — verified via a clean, non-mid-reload screenshot.

9. Automated Regression Suite

9.1 Rationale

Every result above was produced by one-off scripts written, run, and then discarded during this session. That caught real bugs but left no lasting protection against regressions. A permanent Vitest suite was built to close that gap for the General Ledger specifically — the only part of the application that moves real money.

9.2 Architecture

- Integration tests against the real running application and the real development Supabase database (mocking Supabase's query builder would not verify the actual SQL-level guarantees such as atomicity and balance checks)
- Each test file registers its own throwaway tenant (own organization, own auth user) via the same signup routes a real user goes through, and fully tears it down afterward — explicit ordered deletes across every table, not reliance on cascade behavior alone, plus deletion of the Supabase Auth user
- A global setup step reuses an already-running dev server if present, or spawns one automatically

9.3 Test Inventory (18 tests across 6 files)

File	Covers	Tests
journal-posting.test.ts	Balance & atomicity guarantees of manual journal postings	4
auto-posting.test.ts	Exact Dr/Cr correctness for all 4 auto-posting trigger points, plus the paid-invoice void guard	7
period-closing.test.ts	Month lock/unlock; real year-end closing entry computation and exact reversal on reopen; no-op close for an inactive year	3
statements.test.ts	Balance Sheet self-check (Assets = Liabilities + Equity) after a mixed transaction sequence	1
matter-register.test.ts	Batched aggregation correctness for the Matter Register	1
lawyer-overview.test.ts	Utilization-basis switching between logged-ratio and budget-relative	2

9.4 Suite Validity Check

To confirm the suite is not vacuously passing, a real defect was deliberately reintroduced (an off-by-one error in the disbursement posting's debit amount, breaking the balance invariant). The relevant test failed immediately with the exact expected assertion mismatch (`expected 500 to be 200`). The change was then reverted and the full suite re-confirmed green.

9.5 Result

18 / 18 PASS, confirmed repeatable across two consecutive full runs with zero leftover test data in the database afterward (verified by direct query).

10. Conclusion & Recommendations

Every feature phase covered in this report is verified working end-to-end, with all discovered defects fixed and re-confirmed. The General Ledger now carries a permanent, self-cleaning automated test suite that has already proven it can catch a real regression.

Recommended next steps

- Expand the automated suite's tenant-isolation coverage (only one cross-tenant check exists today, on manual journal postings — the pattern could be extended to the other AccountTrack routes).
- Add automated coverage for TimeTrack's billable/non-billable and calendar-linking behavior, which is currently verified only by the manual Playwright pass documented in Section 4.
- Consider wiring the Vitest suite into a CI pipeline once one exists, so it runs on every change rather than only on request.